Bias-Variance Trade-off

Procedure:

- Import the required Python libraries such as NumPy, Matplotlib, Seaborn, and Scikit-learn.
- Create the dataset using the `make_data()` function.
- Fit polynomial regression models with different degrees such as 1, 3, and 5.
- Plot the data and polynomial regression curves.
- Use **validation curves** to compare training and validation scores for different polynomial degrees.
- Generate a larger dataset and repeat the validation curve analysis.
- Compare the training and validation scores to understand **model complexity and the bias-variance trade-off**.

Solution:

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline

def PolynomialRegression(degree=2, **kwargs):
    return make_pipeline(PolynomialFeatures(degree),
                         LinearRegression(**kwargs))

# create some data to which we will fit our model:
import numpy as np

def make_data(N, err=1.0, rseed=1):
    # randomly sample the data
    rng = np.random.RandomState(rseed)
    X = rng.rand(N, 1) ** 2
    y = 10 - 1. / (X.ravel() + 0.1)
    if err > 0:
        y += err * rng.randn(N)
    return X, y

X, y = make_data(50)

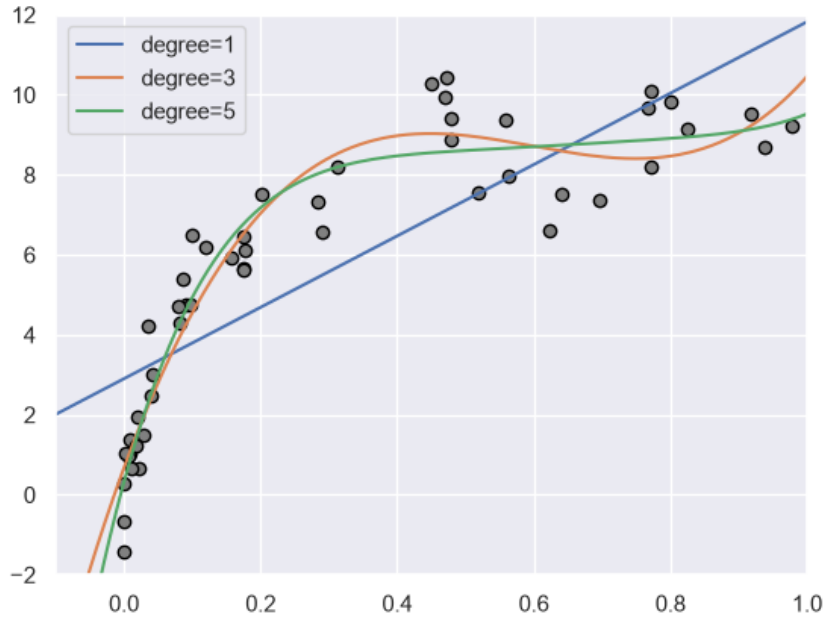
# visualize data, along with polynomial fits of several degrees:

import matplotlib.pyplot as plt
import seaborn; seaborn.set() # plot formatting

X_test = np.linspace(-0.1, 1.1, 500)[:, None]
```

```
plt.scatter(X.ravel(), y, edgecolor='black', color='gray')
axis = plt.axis()

for degree in [1, 3, 5]:
    y_test = PolynomialRegression(degree).fit(X, y).predict(X_test)
    plt.plot(X_test.ravel(), y_test, label='degree={0}'.format(degree))
plt.xlim(-0.1, 1.0)
plt.ylim(-2, 12)
plt.legend(loc='best')
plt.show()
```




```
# This can be done straightforwardly
# using the validation_curve convenience routine provided by Scikit-Learn.

from sklearn.model_selection import validation_curve

degree = np.arange(0, 21)

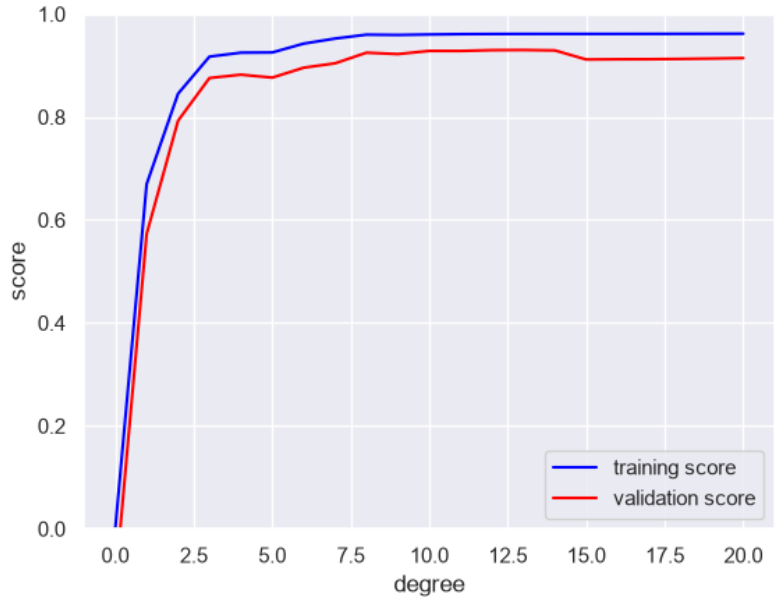

train_score, val_score = validation_curve(PolynomialRegression(), X, y,
param_name='polynomialfeatures__degree',param_range=degree, cv=7)

plt.plot(degree, np.median(train_score, 1), color='blue', label='training score')
plt.plot(degree, np.median(val_score, 1), color='red', label='validation score')
plt.legend(loc='best')

plt.ylim(0, 1)

plt.xlabel('degree')
```

```
plt.ylabel('score')
plt.show()
```

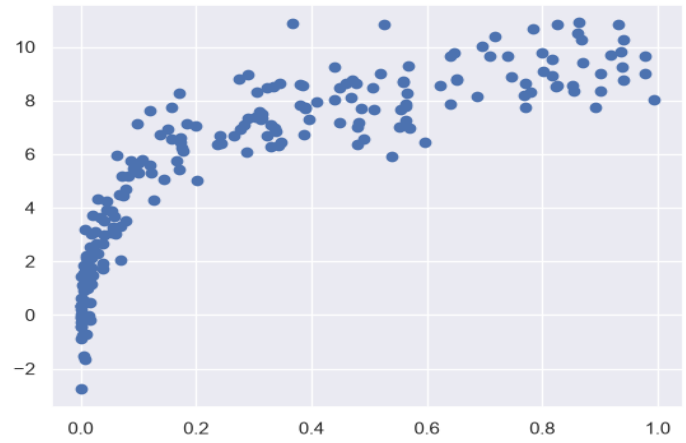



```
# Learning Curves

# One important aspect of model complexity is that the optimal model will generally depend on the
# size of your training data.

# For example, let's generate a new dataset with a factor of five more points:

X2, y2 = make_data(200)
plt.scatter(X2.ravel(), y2)
plt.show()
```



```
# We will duplicate the preceding code to plot the validation curve for this larger dataset;
# for reference let's over-plot the previous results as well:

degree = np.arange(21)
train_score2, val_score2 = validation_curve(PolynomialRegression(), X2, y2,
param_name='polynomialfeatures__degree',param_range=degree, cv=7)

plt.plot(degree, np.median(train_score2, 1), color='blue', label='training score')
plt.plot(degree, np.median(val_score2, 1), color='red', label='validation score')
plt.plot(degree, np.median(train_score, 1), color='blue', alpha=0.3, linestyle='dashed')
plt.plot(degree, np.median(val_score, 1), color='red', alpha=0.3, linestyle='dashed')
plt.legend(loc='lower center')

plt.ylim(0, 1)

plt.xlabel('degree')

plt.ylabel('score')
plt.show()
```

